Salary Prediction Based on Experience

Machine Learning Engineering Project — Beginner Level

Goal: Predict an employee's salary based on their years of experience using Simple Linear Regression.

Steps we'll follow:

1. Generate a synthetic dataset
2. Exploratory Data Analysis (EDA)
3. Data Preprocessing
4. Train a Linear Regression model
5. Evaluate the model
6. Visualize results
7. Predict salary for new inputs

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import warnings
warnings.filterwarnings('ignore')

print('All libraries imported successfully')
```

```
All libraries imported successfully
```

Step 2: Generate Synthetic Dataset

```
import numpy as np
import pandas as pd

np.random.seed(42)


n_samples = 200

experience = np.round(np.random.uniform(0.5, 20.0, n_samples), 1)


salary = 30000 + (4500 * experience) + np.random.normal(0, 5000, n_samples)
salary = np.round(salary, 2)


df = pd.DataFrame({
    'YearsExperience': experience,
    'Salary': salary
})

print(f'Dataset Shape: {df.shape}')
print()
print('First 10 rows:')
df.head(10)
```

```
Dataset Shape: (200, 2)

First 10 rows:
```

	YearsExperience	Salary
0	7.8	61699.88
1	19.0	116661.27
2	14.8	98065.36
3	12.2	81328.24
4	3.5	55078.87
5	3.5	48119.16
6	1.6	31243.48
7	17.4	111582.77
8	12.2	80026.59
9	14.3	98285.42

Next steps: New interactive sheet

Step 3: Exploratory Data Analysis (EDA)

```
print('Dataset Info')
print(df.info())
print()
print('Statistical Summary')
df.describe().round(2)
```

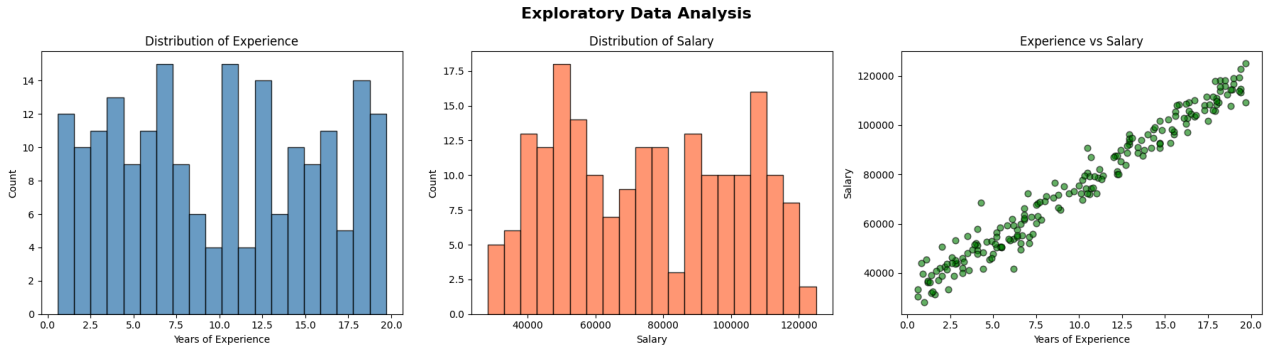
```
Dataset Info
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  200 non-null    float64
 1   Salary           200 non-null    float64
dtypes: float64(2)
memory usage: 3.3 KB
None

Statistical Summary
```

	YearsExperience	Salary
count	200.00	200.00
mean	9.93	75035.57
std	5.75	26219.59
min	0.60	28195.58
25%	4.97	51773.27
50%	10.15	72720.67
75%	15.22	98120.38
max	19.70	125038.32

```
print('Missing Values')
print(df.isnull().sum())
print()
print('No missing values — dataset is clean')
```

```
Missing Values
YearsExperience    0
Salary             0
dtype: int64

No missing values — dataset is clean
```

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('Exploratory Data Analysis', fontsize=16, fontweight='bold')


axes[0].hist(df['YearsExperience'], bins=20, color='steelblue', edgecolor='black', alpha=0.8)
axes[0].set_title('Distribution of Experience')
axes[0].set_xlabel('Years of Experience')
axes[0].set_ylabel('Count')


axes[1].hist(df['Salary'], bins=20, color='coral', edgecolor='black', alpha=0.8)
axes[1].set_title('Distribution of Salary')
axes[1].set_xlabel('Salary')
axes[1].set_ylabel('Count')


axes[2].scatter(df['YearsExperience'], df['Salary'], color='green', alpha=0.6, edgecolor='black', s=40)
axes[2].set_title('Experience vs Salary')
axes[2].set_xlabel('Years of Experience')
axes[2].set_ylabel('Salary')

plt.tight_layout()
plt.show()
```




```
correlation = df['YearsExperience'].corr(df['Salary'])
print(f'Correlation between Experience and Salary: {correlation:.4f}')
print('(Value close to 1 = strong positive relationship)')
```

```
Correlation between Experience and Salary: 0.9828
(Value close to 1 = strong positive relationship)
```

Step 4: Data Preprocessing & Train-Test Split

```
X = df[['YearsExperience']]
y = df['Salary']


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f'Training set size  : {X_train.shape[0]} samples')
print(f'Testing  set size  : {X_test.shape[0]} samples')
print()
print('Data split complete')
```

```
Training set size  : 160 samples
Testing  set size  : 40 samples

Data split complete
```

Step 5: Train the Linear Regression Model

```
model = LinearRegression()
model.fit(X_train, y_train)

print('Model trained successfully')
print()
print(f'Model Equation:')
print(f'Salary = {model.coef_[0]:.2f} × YearsExperience + {model.intercept_:.2f}')
print()
print(f'Interpretation:')
print(f'  For every 1 year of experience, salary increases by ₹{model.coef_[0]:,.2f}')
print(f'   A fresher (0 years) would earn approximately ₹{model.intercept_:,.2f}')
```

```
Model trained successfully

Model Equation:
Salary = 4526.41 × YearsExperience + 30063.68

Interpretation:
  For every 1 year of experience, salary increases by ₹4,526.41
   A fresher (0 years) would earn approximately ₹30,063.68
```

Step 6: Evaluate the Model

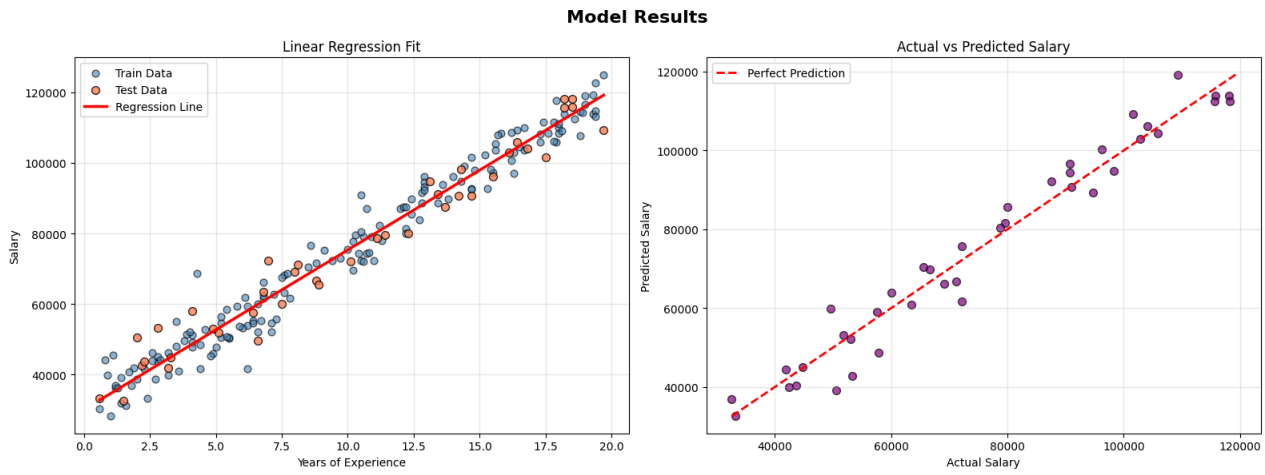
```
y_pred = model.predict(X_test)


mae  = mean_absolute_error(y_test, y_pred)
mse  = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2   = r2_score(y_test, y_pred)

print('Model Evaluation Metrics')
print(f'  MAE  (Mean Absolute Error)       : ₹{mae:,.2f}')
print(f'  MSE  (Mean Squared Error)        : {mse:,.2f}')
print(f'  RMSE (Root Mean Squared Error)   : ₹{rmse:,.2f}')
print(f'  R²   (R-Squared Score)           : {r2:.4f}  ({r2*100:.2f}%)')
print()
print(' R² Score Explanation:')
print(f' model explains {r2*100:.1f}% of the variance in salaries')
```

```
Model Evaluation Metrics
  MAE  (Mean Absolute Error)       : ₹4,193.21
  MSE  (Mean Squared Error)        : 27,106,326.69
  RMSE (Root Mean Squared Error)   : ₹5,206.37
  R²   (R-Squared Score)           : 0.9572  (95.72%)

 R² Score Explanation:
 model explains 95.7% of the variance in salaries
```

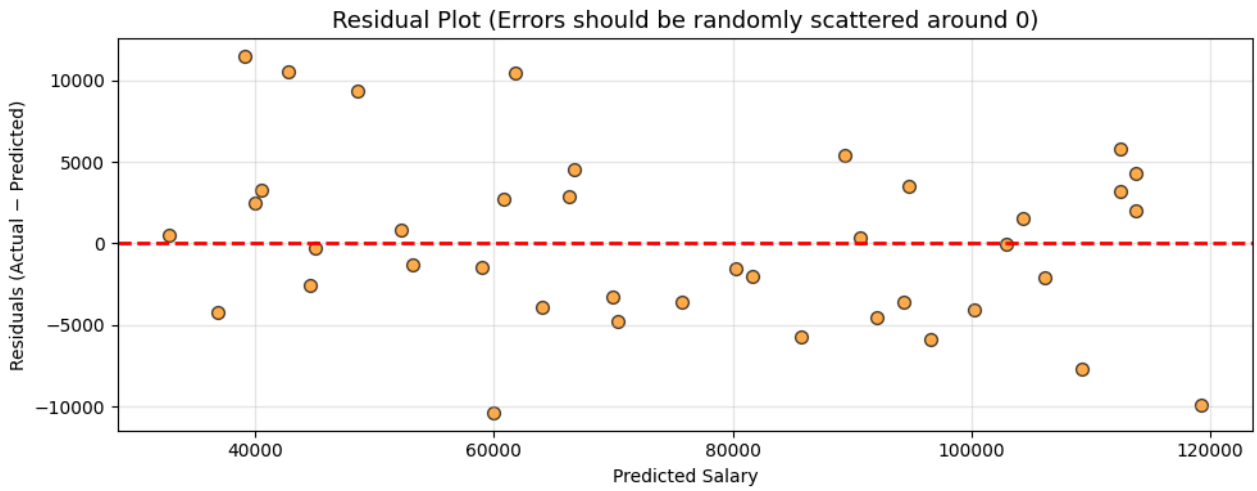
Step 7: Visualize Results

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Model Results', fontsize=16, fontweight='bold')


axes[0].scatter(X_train, y_train, color='steelblue', alpha=0.6, label='Train Data', edgecolor='black', s=40)
axes[0].scatter(X_test,  y_test,  color='coral',     alpha=0.8, label='Test Data',  edgecolor='black', s=50)


x_line = np.linspace(df['YearsExperience'].min(), df['YearsExperience'].max(), 100).reshape(-1, 1)
y_line = model.predict(x_line)
axes[0].plot(x_line, y_line, color='red', linewidth=2.5, label='Regression Line')

axes[0].set_title('Linear Regression Fit')
axes[0].set_xlabel('Years of Experience')
axes[0].set_ylabel('Salary')
axes[0].legend()
axes[0].grid(True, alpha=0.3)


axes[1].scatter(y_test, y_pred, color='purple', alpha=0.7, edgecolor='black', s=50)
min_val = min(y_test.min(), y_pred.min())
max_val = max(y_test.max(), y_pred.max())
axes[1].plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2, label='Perfect Prediction')
axes[1].set_title('Actual vs Predicted Salary')
axes[1].set_xlabel('Actual Salary')
axes[1].set_ylabel('Predicted Salary')
axes[1].legend()
```

```
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```




```
residuals = y_test - y_pred

plt.figure(figsize=(10, 4))
plt.scatter(y_pred, residuals, color='darkorange', alpha=0.7, edgecolor='black', s=50)
plt.axhline(y=0, color='red', linestyle='--', linewidth=2)
plt.title('Residual Plot (Errors should be randomly scattered around 0)', fontsize=13)
plt.xlabel('Predicted Salary')
plt.ylabel('Residuals (Actual - Predicted)')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('Residuals scattered randomly → model assumptions are satisfied')
```




```
Residuals scattered randomly → model assumptions are satisfied
```

Step 8: Predict Salary for New Inputs

```
test_cases = [1, 3, 5, 7, 10, 15]

print('Salary Predictions')
print(f'{"Experience (Years)":<22} {"Predicted Salary"}')
print('-' * 40)

for exp in test_cases:
    predicted = model.predict([[exp]])[0]
    print(f'{exp:<22} ₹{predicted:,.2f}')
```

```
Salary Predictions
Experience (Years)     Predicted Salary
----------------------------------------
1                      ₹34,590.09
3                      ₹43,642.91
5                      ₹52,695.73
7                      ₹61,748.55
10                     ₹75,327.78
15                     ₹97,959.83
```